Parameterized Verification

Abstract

We consider the model checking problem for *parameterized systems*: systems that consist of an arbitrary number of components. The task is to show correctness regardless of the number of components. Examples of parameterized systems include mutual exclusion algorithms, cache coherence protocols, network protocols, and distributed algorithms. The problem is undecidable in general, but a wide range of techniques has been developed that achieve decidability or effective automation for practically important classes of systems.

We present three techniques in detail. Regular model checking represents configurations as words and transition relations as finite-state transducers, and we describe an acceleration technique that computes the transitive closure of a transducer by quotienting. Monotonic abstraction over-approximates systems with global conditions by well-structured transition systems, so that safety can be checked by backward reachability analysis on upward-closed sets. View abstraction inspects only small instances of the system and recombines their views to obtain an over-approximation of all instances, detecting a cutoff during the analysis; it is complete for a large class of well-structured systems. We then give an overview of other approaches, including counting abstraction, cutoff theorems, network invariants, deductive and SMT-based methods, and applications to programs running under weak memory models and to parameterized quantum circuits.

1 Introduction

The behavior of many computer systems is described by one or more *parameters*, each of which ranges over a given domain. The parameters may relate to the size or topology of a network, to the data types over which a system is built, or to the initial values of variables. The *parameterized verification problem* is to prove or refute that a specification holds for all values of the parameters. In this chapter, we concentrate on a particular class of parameterized systems, namely systems that consist of an arbitrary number of components, or processes. The size of the system is then a parameter of the verification problem, and parameterized verification amounts to proving correctness regardless of the number of processes.

Parameterized systems appear naturally in many applications, such as mutual exclusion algorithms, bus protocols, network protocols, cache coherence protocols, distributed algorithms, and web services. A mutual exclusion protocol must guarantee that no two processes are in their critical sections at the same time, whatever the number of processes that participate in a session of the protocol. A cache coherence protocol must keep the shared data consistent for any number of caches. Sensor networks may consist of thousands of nodes, and network protocols may involve

millions of participants. Classical model checking verifies one instance of such a system at a time, for a fixed number of processes. Even if each process is finite-state, the family of all instances has an infinite state space, so there is a need to extend verification from individual instances to the parameterized case.

Parameterized verification is undecidable in general [AK86, Suz88], even when each process is finite-state. Research in the area has therefore identified classes of systems for which the problem is decidable, developed algorithms that are not guaranteed to terminate but that terminate often in practice, and used abstractions that over-approximate the behavior of the system. A recurring theme is the need for a finite symbolic representation of infinite sets of configurations, such as regular languages or upward-closed sets. Another recurring theme is *monotonicity*: the observation that adding processes to a system often only enables more behavior, which is exploited by cutoff arguments and by the theory of well-structured transition systems. A third theme is the observation that the behavior of a large instance is often already visible in a small one. *Cutoff* methods make this precise: a static cutoff theorem shows that, for a given class of systems, correctness for all instances up to a fixed size implies correctness for all sizes [EN95, EK00], while dynamic cutoff methods discover such a size during the analysis [AHH16, KKW10].

This chapter is a revision of the corresponding chapter in the first edition of this handbook [AST18]. Instead of surveying all approaches at the same level of detail, we present three techniques in depth, with their algorithms and worked examples, and give an overview of the other approaches. The chapter is organized as follows. Section 2 introduces parameterized systems and the verification problem, and recalls the basic notions of well-quasi-orderings and backward reachability. Section 3 describes the early developments of the field. Section 4 presents *regular model checking* [KMM+97, KMM+01, BJNT00], which represents configurations as words and transition relations as finite-state transducers, and describes an acceleration technique for computing transitive closures of transducers. Section 5 presents *monotonic abstraction* [ADBHR07], which over-approximates systems with global conditions by well-structured transition systems, so that safety can be checked by backward reachability analysis. Section 6 presents *view abstraction* [AHH13, AHH16], a forward analysis that inspects only small instances of the system and detects a cutoff during the analysis. Section 7 gives an overview of other approaches, including counting abstraction, cutoff theorems, network invariants and other abstractions, deductive and SMT-based methods, and two recent applications, namely programs running under weak memory models and parameterized quantum circuits. Section 8 discusses industrial applications, Section 9 relates this chapter to other chapters of the handbook, and Section 10 concludes with open problems. Throughout, we concentrate on safety properties, and mention liveness properties only occasionally.

2 Parameterized Systems

In this section, we define parameterized systems with a linear topology and the parameterized reachability problem. Other topologies and communication primitives are introduced in the sections where they are used.

2.1 Model

For a finite set A, we write A^* for the set of finite words over A and A^+ for the set of nonempty words. For a word c, we write $|c|$ for its length and $c[i]$ for its ith letter, for $1 \leq i \leq |c|$.

A *parameterized system* is a pair $\mathcal{P} = (Q, \Delta)$, where Q is a finite set of *local states* and Δ is a finite set of *transition rules*. A rule is either local or global. A *local* rule has the form $s \to d$, where $s, d \in Q$. A *global* rule has the form

$$\textbf{if } \mathsf{Q}\, j \sim i : S \textbf{ then } s \to d,$$

where $\mathsf{Q} \in \{\forall, \exists\}$ is the *quantifier*, $\sim \in \{<, >, \neq\}$ is the *range*, and $S \subseteq Q$. We call s the *source* and d the *destination* of the rule. A rule with quantifier $\forall$ is *universal*, and a rule with quantifier $\exists$ is *existential*.

A *configuration* of $\mathcal{P}$ is a word $c \in Q^+$, where $c[i]$ is the local state of process i. The number $|c|$ of processes is arbitrary but does not change during a run. Let c be a configuration, δ a rule with source s and destination d, and i a position with $c[i] = s$. We write $\delta(c, i)$ for the configuration c' with $c'[i] = d$ and $c'[j] = c[j]$ for all $j \neq i$, provided that one of the following conditions holds:

- δ is local;
- δ is universal, and $c[j] \in S$ for all positions $j \sim i$;
- δ is existential, and $c[j] \in S$ for some position $j \sim i$.

Otherwise, $\delta(c, i)$ is undefined. We call process i the *active* process, and the processes j with $j < i$ and $j > i$ its *left* and *right context*. We write $c \to c'$ if $c' = \delta(c, i)$ for some rule $\delta \in \Delta$ and position i, and $\to^*$ for the reflexive transitive closure of $\to$. For a set X of configurations, we write $Post(X)$ for the set of configurations c' with $c \to c'$ for some $c \in X$, and $Pre(X)$ for the set of configurations c with $c \to c'$ for some $c' \in X$.

L: Is process creation excluded? Should we say it explicitly?

2.2 The Reachability Problem

An instance of the *parameterized reachability problem* is given by a parameterized system $\mathcal{P}$, a set $Init \subseteq Q^+$ of *initial* configurations, and a set $Bad \subseteq Q^+$ of *bad* configurations. A configuration c is *reachable* if $c_0 \to^* c$ for some $c_0 \in Init$. We write $\mathcal{R}$ for the set of reachable configurations. The system is *safe* if $\mathcal{R} \cap Bad = \emptyset$. Typically, $Init$ is a regular set, such as the set of configurations in which all processes are in a given initial state, and Bad is described by a finite set of patterns, such as the configurations in which two processes are in their critical sections.

The problem is undecidable, even for finite-state processes. Apt and Kozen [AK86] showed that verifying a specification for all sizes is not even semi-decidable, and Suzuki [Suz88] showed that the problem remains undecidable for unidirectional rings of identical finite-state processes. We sketch a reduction from the halting problem in Section 4.

3 Early Developments

The questions studied in this chapter were first formulated in a body of work from the late 1980s and 1990s that established both the model and the basic techniques of parameterized verification. This section reviews that work and some of its later developments.

Correspondence between instances. While the undecidability results of Section 2 showed early on that a general algorithm cannot exist [AK86, Suz88], Browne, Clarke, and Grumberg [BCG89] proposed a way of transferring verification results from small instances to large ones. They introduced an indexed version of the temporal logic CTL* without the next-time operator, together with a notion of *correspondence* between systems of different sizes that preserves the formulas of this logic. Once a correspondence between a small instance and all larger instances has been established, model checking the small instance suffices. Establishing the correspondence requires, in general, an argument by the user, but the approach anticipates the cutoff theorems discussed in Section 7, in which the transfer from small to large instances is guaranteed by the structure of the systey it explicitly?paragraphSystems with many processes. A foundational formulation considers a system composed of a single control process together with an unbounded collection of user processes sharing a common definition, communicating through rendez-vous synchronization in the style of CCS [GS92]. The central contribution was to show that reasoning about the entire infinite family can be reduced to a finite analysis: decision procedures were given for checking whether every process satisfies a given indexed temporal specification. When all processes are identical, the procedure runs in time polynomial in the size of the process template; in the presence of a control process, it runs in doubly exponential time. The reachability questions arising in the analysis were connected to the theory of Petri nets and vector addition systems, through a *counting abstraction*: since the user processes are anonymous, it suffices to record how many of them are in each local state. The question of whether some user process can reach a given local state becomes the *coverability* problem for Petri nets, which is decidable and, by the results of Lipton and Rackoff, ExpSpace-complete [Rac78, Esp98]. This early demarcation of the boundary between decidable and undecidable cases, and of the algebraic structures underlying it, shaped much of the work that followed.

L: or, whether some user processes can reach a specific combination of states?

Induction over the number of processes. A second early line of work used induction on the number of processes. Kurshan and McMillan [KM89], and independently Wolper and Lovinfosse [WL89], observed that if a process I is found such that I *abstracts* a single copy of the template, in a sense that preserves the properties of interest, and the composition of I with one more copy is again abstracted by I, then I abstracts the composition of any number of copies. The process I is then a *network invariant*,

and verifying the composition of I with a fixed environment is sufficient for all sizes. We return to this idea in Section 7. Clarke, Grumberg, and Jha [CGJ97] generalized the setting to families of networks generated by *network grammars*, using abstraction to make the search for invariants effective.

A complementary line of work targeted linear networks of arbitrary size and introduced automata as inductive invariants [SG99]. A correctness property is itself specified by an automaton, from which an invariant automaton is synthesized; an induction on the number of processes then establishes that the property is preserved as the network grows. The method was shown to be complete for safety properties. It directly anticipates both the network-invariant approach of Section 7 and the regular model checking framework of Section 4, in which sets of global configurations of unbounded width are represented symbolically by finite automata.

Symmetry. A further theme running through this early work is the exploitation of the symmetry inherent in a population of identical processes. Because the identity of an individual process is immaterial to the global behavior of the system, verification can be carried out over equivalence classes of configurations rather than over configurations themselves [ES96, CEFJ96, ID96]. Two configurations are equivalent if one is obtained from the other by permuting the process indices, and the quotient of the state space under this equivalence can be exponentially smaller than the original. Symmetry reduction applies to each instance separately, so it does not by itself solve the parameterized problem, but it explains why configurations of symmetric systems can be represented by multisets, and it is the basis of the counting abstraction above. Combining symmetry with fairness leads to on-the-fly model checking procedures that keep the explored state space manageable [GS97].

L: i somehow don't get the previous sentence

Broadcast protocols and cache coherence. Esparza, Finkel, and Mayr [EFM99] studied *broadcast protocols*, in which a process can send a message that is received by all other processes in the same step. Broadcast protocols are an extension of Petri nets, in which a transition can move all tokens from one place to another. Safety properties expressed by upward-closed sets remain decidable for broadcast protocols, and can be checked by backward reachability analysis (Section 5.3), while liveness properties become undecidable. Delzanno [Del00] applied this framework to cache coherence protocols. Using the counting abstraction and symbolic representations based on linear arithmetic constraints, he verified several coherence protocols automatically for an arbitrary number of caches. This was one of the first demonstrations that parameterized verification could handle protocols of practical relevance.

Later developments. These ideas have been extended in many directions. Parameterized systems with pairwise rendezvous have been analyzed on a range of topologies [AKR$^+$18]. In

asynchronous shared-memory systems, an unbounded number of anonymous processes communicate through shared registers without locks [EGM16]. In *population protocols*, anonymous agents interact in pairs to compute a predicate of their initial states [AAD+06], and verifying that a protocol computes the intended predicate for every population size is a parameterized problem that can be reduced to problems on Petri nets [EGLM17]. In *ad hoc networks*, processes communicate by broadcast to their neighbors in an unknown graph; the problem is undecidable in general, and becomes decidable under restrictions on the possible topologies [DSZ10]. A recurring observation in this line of work is that, when processes are anonymous and any behavior of one process can be replicated by others, the complexity of parameterized verification is often much lower than that of Petri net coverability [Esp14].

Taken together, these strands form the conceptual foundation on which the remainder of this chapter builds: finite reductions for synchronizing populations, induction over the number of processes, automata-based invariants for linear networks, symmetry reduction for replicated components, and well-structured models of broadcast.

4 Regular Model Checking

In this section, we consider parameterized systems with a linear topology in which the processes are finite-state and a process may communicate with its immediate neighbors. Linear topologies are common in parameterized systems, where the position of a process may, for instance, reflect its priority compared to the other processes. Already for this class of systems, all non-trivial verification problems are undecidable. We present the framework of *regular model checking* [KMM+97, KMM+01, WB98, BJNT00], which uses regular languages as a uniform representation of sets of configurations, and we describe in detail one technique within the framework, namely *acceleration* based on quotienting [AJNd02, AJNd03].

4.1 Near-Neighbor Communication

We consider systems that consist of an arbitrary number of finite-state processes organized in a linear array. In a *near-neighbor* communication, a process communicates with its left or right neighbor, and the two processes change their states simultaneously. As an example, we use a simple *token-passing protocol*. Initially, the leftmost process owns the token. In each step, the process that has the token passes it to its right neighbor. The safety property states that at most one process has the token at any point during the execution of the system.

Near-neighbor communication leads to undecidability of safety properties. The proof is a reduction from the halting problem for Turing machines, in a similar manner to the proof for rings [Suz88]. Given a Turing machine $\mathcal{M}$, the instance of size n of the system simulates the computations of $\mathcal{M}$ that use at most n tape cells. Each cell is represented by one process, whose local state encodes the symbol stored in the cell, together with a flag

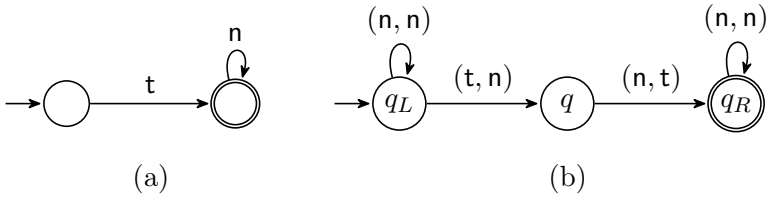


Figure 1: The token-passing protocol. (a) An automaton for the initial configurations tn^*. (b) A transducer T for the transition relation.

that indicates whether the head of $\mathcal{M}$ points to the cell. If the flag is set, the local state also stores the control state of $\mathcal{M}$. Near-neighbor communication is then used to move the head, to change the symbol in the current cell, and to change the control state. A bad configuration is reachable for some n if and only if $\mathcal{M}$ halts.

4.2 Words, Relations, and Transducers

In regular model checking, the behavior of a system is described in three steps. First, we fix a finite alphabet in which each symbol represents the local state of a process. Second, a configuration is represented by a word, where each position describes the state of one process, and sets of configurations are represented by finite automata or regular expressions. Third, the transition relation is represented by a finite automaton that reads pairs of symbols, a so-called *finite-state transducer*. The framework makes it possible to use the well-developed algorithms for manipulating regular languages, which have been implemented, for example, in the Mona tool [HJJ$^+$95].

Let Σ be a finite alphabet. We consider *length-preserving* relations on words, that is, relations $R \subseteq \Sigma^* \times \Sigma^*$ such that u and v have the same length whenever $(u, v) \in R$. For a set $L \subseteq \Sigma^*$ of words, we write $L \circ R$ for the set of words v such that $(u, v) \in R$ for some $u \in L$. For two relations R_1 and R_2, we write $R_1 \circ R_2$ for their composition, which contains the pairs (u, w) such that $(u, v) \in R_1$ and $(v, w) \in R_2$ for some word v. We define R^0 to be the identity relation and $R^{i+1} = R^i \circ R$. The transitive closure of R is the union R^+ of all R^i with $i \geq 1$, and the reflexive transitive closure is $R^* = R^0 \cup R^+$.

A *transducer* over Σ is a tuple $T = (P, q_0, \Theta, F)$, where P is a finite set of states, $q_0 \in P$ is the initial state, $\Theta \subseteq P \times (\Sigma \times \Sigma) \times P$ is the transition relation, and $F \subseteq P$ is the set of accepting states. We write $q \xrightarrow{(a,b)} r$ if $(q, (a, b), r) \in \Theta$. A transducer is a finite automaton over the alphabet $\Sigma \times \Sigma$, so it accepts words of pairs $(a_1, b_1)(a_2, b_2) \cdots (a_n, b_n)$. We say that T *accepts* the pair of words $(a_1 \cdots a_n, b_1 \cdots b_n)$ if it accepts the word $(a_1, b_1) \cdots (a_n, b_n)$, and we write $R(T)$ for the length-preserving relation that consists of all pairs accepted by T. In regular model checking, $R(T)$ is the transition relation of the system. For simplicity, we write $R^i(T)$, $R^+(T)$, and $R^*(T)$ for $(R(T))^i$, $(R(T))^+$, and $(R(T))^*$. A pair (u, v) belongs to $R^i(T)$ if u can be rewritten into v by i successive runs of T.

L: Note that these are (letter-to-letter/length

The token-passing protocol. For the token-passing protocol, the alphabet is $\Sigma = \{\mathsf{t}, \mathsf{n}\}$, where t means that the process has the token and n means that it does not. For instance, the word nntnn represents a configuration with five processes in which the third process has the token. The set of initial configurations is given by the regular expression tn^* (Figure 1(a)): the leftmost process has the token, and it is followed by an arbitrary number of processes without the token. The transition relation is given by the transducer T of Figure 1(b). The transducer copies a prefix of processes without the token in state q_L, reads the pair (t, n) for the process that gives the token away and the pair (n, t) for its right neighbor, and copies the remaining processes in state q_R. For instance, T accepts the word $(\mathsf{n}, \mathsf{n})(\mathsf{n}, \mathsf{n})(\mathsf{t}, \mathsf{n})(\mathsf{n}, \mathsf{t})(\mathsf{n}, \mathsf{n})$, which represents the pair $(\mathsf{nntnn}, \mathsf{nnntn})$, in which the token is passed from the third to the fourth process.

Safety. Let *Init* be the regular set of initial configurations and *Bad* the regular set of bad configurations. A safety property is verified by checking that no bad configuration is reachable, that is, that

$$(\mathit{Init} \circ R^*(T)) \cap \mathit{Bad} = \emptyset.$$

For the token-passing protocol, the bad configurations are those in which at least two processes have the token, given by the regular expression $(\mathsf{t}+\mathsf{n})^*\mathsf{t}(\mathsf{t}+\mathsf{n})^*\mathsf{t}(\mathsf{t}+\mathsf{n})^*$. Since $R^*(T) = R^0 \cup R^+(T)$, it suffices to compute a transducer T^+ with $R(T^+) = R^+(T)$. The reachable set $\mathit{Init} \circ R^*(T)$ is then regular and can be computed with standard automata constructions, and the intersection with *Bad* can be checked for emptiness. The transitive closure is also the key to verifying liveness properties, where it is used to detect cycles in the transition relation [AJN+04].

4.3 Acceleration

In general, the transitive closure $R^+(T)$ is not computable, and it may not even be regular. Our goal is a technique that computes a finite-state transducer for $R^+(T)$ in the cases where such a transducer exists. A first attempt is to compute, for $n = 1, 2, 3, \ldots$, a transducer T^n with $R(T^n) = R^n(T)$. For the token-passing protocol, T^3 passes the token three positions to the right (Figure 2(a)), and the transducer for $R^+(T)$ passes the token an arbitrary positive number of positions (Figure 2(b)). Computing the approximations T^n one by one does not converge, since T^n needs $n + 2$ states. Instead, we *accelerate* the computation in two steps. First, we define an infinite-state transducer, the *column transducer*, that accepts $R^+(T)$. We never build the column transducer explicitly. Second, we define an equivalence relation on its states and construct the quotient with respect to this relation incrementally. If the construction terminates, the result is a finite-state transducer for $R^+(T)$.

The column transducer. The column transducer simulates an arbitrary positive number i of successive runs of T. Each of its transitions combines i transitions of T operating at the same position in the word. The first transition rewrites the original

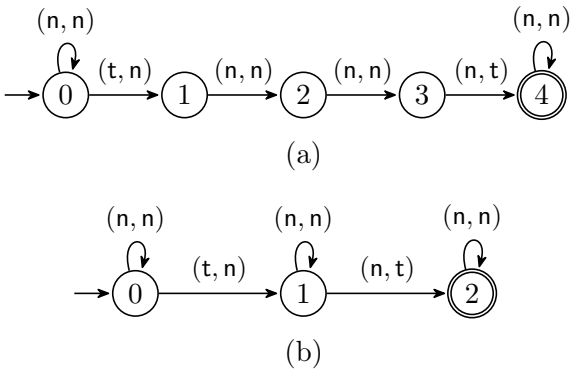


Figure 2: (a) The transducer T^3, which passes the token three positions to the right. (b) A transducer for the transitive closure $R^+(T)$.

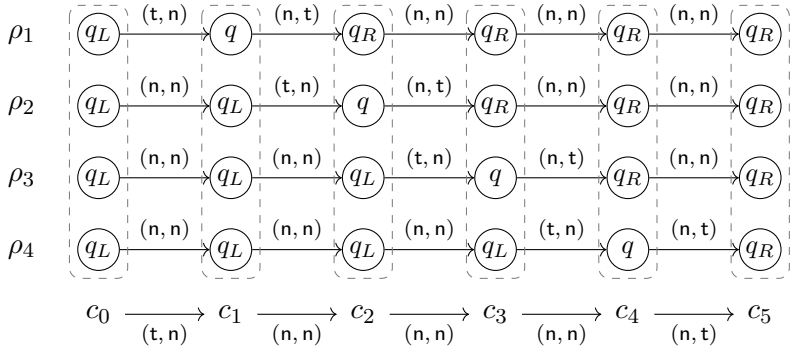


Figure 3: Four runs $\rho_1, \ldots, \rho_4$ of the transducer T of Figure 1(b). Read column by column, they form a single run $c_0 c_1 \cdots c_5$ of the column transducer, shown at the bottom with the labels of its steps.

symbol a_0 to a_1, the second rewrites a_1 to a_2, and so on, until the i-th transition produces a_i. Thus, a_j is the symbol at that position after the j-th run of T.

Figure 3 shows four runs $\rho_1, \ldots, \rho_4$ of the transducer T on words of length five. The runs relate the pairs $(\mathsf{tnnnn}, \mathsf{ntnnn})$, $(\mathsf{ntnnn}, \mathsf{nntnn})$, $(\mathsf{nntnn}, \mathsf{nnntn})$, and $(\mathsf{nnntn}, \mathsf{nnnnt})$, so the pair $(\mathsf{tnnnn}, \mathsf{nnnnt})$ belongs to $R^4(T)$. If we read the four runs column by column, we obtain a single run whose states, called *columns*, are sequences of four states of T. For instance, the first step is labeled (t, n), since the first steps of the four runs rewrite t to n, and then n to n three times. The second step is labeled (n, n), since the second steps rewrite n to t, then t to n, and then n to n twice.

Formally, from a transducer $T = (P, q_0, \Theta, F)$ we define the *column transducer* T^{col} as follows. Its states are the nonempty sequences of states of T, that is, the set P^+. Its initial states are the sequences q_0^i with $i \geq 1$, and its accepting states are the nonempty sequences of accepting states of T. For columns $c = q_1 \cdots q_m$ and $c' = r_1 \cdots r_m$ of the same length, there is a transition $c \xrightarrow{(a,a')} c'$ if there are symbols $a_0, a_1, \ldots, a_m$ with $a_0 = a$ and $a_m = a'$ such that $q_j \xrightarrow{(a_{j-1}, a_j)} r_j$ is a transition of T for each j with $1 \leq j \leq m$. A run of the column transducer from q_0^i to a sequence in F^i accepts exactly the pairs in $R^i(T)$, so T^{col} accepts the relation $R^+(T)$. Its set of states is infinite, however,

so it cannot be built explicitly.

Quotienting. We now define an equivalence relation on columns that identifies columns which differ only in the number of repetitions of certain states. We say that a state q of T is *left-copying* if every run of T from the initial state to q reads only pairs of the form (a, a). Symmetrically, q is *right-copying* if every run from q to an accepting state reads only pairs of the form (a, a). In other words, the prefix read before a left-copying state and the suffix read after a right-copying state are copied unchanged. In Figure 1(b), the state q_L is left-copying and the state q_R is right-copying. We say that two columns are *equivalent* if they become equal when every block of consecutive repetitions of a left-copying or right-copying state is replaced by a single occurrence of that state. For instance, the columns $q_R q_R q\, q_L$ and $q_R q\, q_L q_L$ are equivalent. Each equivalence class is described by a regular expression $e_1 e_2 \cdots e_k$, where each e_j is either q^+ for a left-copying or right-copying state q, or a single state that is neither left-copying nor right-copying, and where two consecutive e_j are equal only if they are single states. We write $[c]$ for the equivalence class of a column c.

The *quotient transducer* has the equivalence classes as states. Its initial state is $[q_0] = q_0^+$, assuming that q_0 is left-copying, and its accepting states are the classes of accepting columns. There is a transition $[c] \xrightarrow{(a,a')} [c']$ whenever $c \xrightarrow{(a,a')} c'$ is a transition of the column transducer. With this choice of equivalence, the quotient accepts the same relation as the column transducer [AJNd02], namely $R^+(T)$. The quotient may still be infinite, since $R^+(T)$ need not be regular, but in many practical cases it is finite.

The procedure. The quotient is built incrementally by Procedure 1, which explores the classes reachable from the initial class. To compute the successors of a class, we use two operations. For a state q, we write $q^\oplus$ for q^+ if q is left-copying or right-copying, and for q otherwise. For two classes, we write $[c] \odot [c'] = [c \cdot c']$ for the class of the concatenated column; on regular expressions, $(e_1 \cdots e_k) \odot (f_1 \cdots f_l)$ is $e_1 \cdots e_k f_1 \cdots f_l$, except when e_k and f_1 are both equal to q^+ for some copying state q, in which case it is $e_1 \cdots e_k f_2 \cdots f_l$. We write $C \Rightarrow_{(a,a')} D$ for classes C and D if either

- $q \xrightarrow{(a,a')} r$ is a transition of T, with $C = q^\oplus$ and $D = r^\oplus$, or
- $C = C_1 \odot C_2$ and $D = D_1 \odot D_2$, and there is a symbol b with $C_1 \Rightarrow_{(a,b)} D_1$ and $C_2 \Rightarrow_{(b,a')} D_2$.

The second case composes the effect of the upper part of a column with that of its lower part, as in Figure 3.

The procedure maintains a set W of classes that have been found to be reachable but whose successors have not yet been computed. It starts from the initial class q_0^+. In each iteration, it removes a class from W and, if the class has not been explored before, adds its outgoing transitions and its successors. The relation accepted by the partial result grows with each iteration and is always contained in $R^+(T)$. The procedure terminates when

Procedure 1 Acceleration by quotienting

Require: A transducer $T = (P, q_0, \Theta, F)$
Ensure: A transducer (S, q_0^+, Θ', F') for $R^+(T)$, if the procedure terminates

```
1: S ← ∅; Θ' ← ∅; F' ← ∅; W ← {q_0^+}
2: while W ≠ ∅ do
3:    pick and remove a class C from W
4:    if C ∉ S then
5:       S ← S ∪ {C}
6:       for all a, a', D with C ⇒_(a,a') D do
7:          W ← W ∪ {D}
8:          Θ' ← Θ' ∪ {(C, (a, a'), D)}
9:          if D consists of accepting columns then
10:            F' ← F' ∪ {D}
11:         end if
12:      end for
13:   end if
14: end while
```

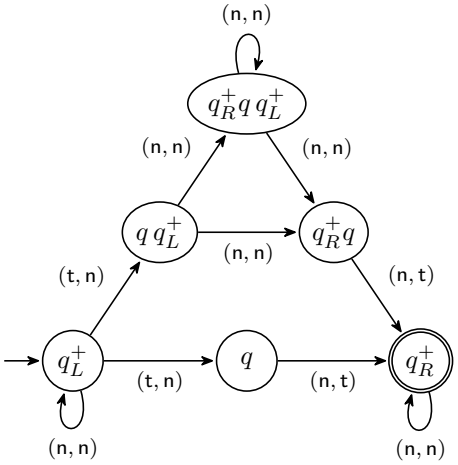


Figure 4: The transducer for $R^+(T)$ computed by Procedure 1 for the token-passing protocol. Each state is an equivalence class of columns.

W is empty, which happens exactly when the reachable part of the quotient is finite.

The token-passing protocol. Figure 4 shows the result of the procedure on the transducer of Figure 1(b). The procedure starts with the class q_L^+. Since $q_L \xrightarrow{(\mathsf{t},\mathsf{n})} q$ and $q_L \xrightarrow{(\mathsf{n},\mathsf{n})} q_L$, we obtain the transition $q_L^+ \Rightarrow_{(\mathsf{t},\mathsf{n})} q$, and, by composing the two, the transition $q_L^+ = q_L^+ \odot q_L^+ \Rightarrow_{(\mathsf{t},\mathsf{n})} q \odot q_L^+ = q\, q_L^+$. The loop on q_L^+ comes from the loop on q_L. From q, the transition $q \xrightarrow{(\mathsf{n},\mathsf{t})} q_R$ gives the accepting class q_R^+, which has a loop labeled (n,n). From the class $q\, q_L^+$, the upper part writes the token, since $q \Rightarrow_{(\mathsf{t},\mathsf{n})} q_R^+$, and the lower part reads it, since $q_L^+ \Rightarrow_{(\mathsf{t},\mathsf{n})} q$ and $q_L^+ \Rightarrow_{(\mathsf{t},\mathsf{n})} q\, q_L^+$. Composing them gives the transitions from $q\, q_L^+$ to $q_R^+ q$ and to $q_R^+ q\, q_L^+$, both labeled (n,n). From $q_R^+ q$, the lower state q writes the token, which gives the transition to q_R^+ labeled (n,t). Finally, the class $q_R^+ q\, q_L^+$ has a loop and a transition to $q_R^+ q$, both

labeled (n, n). At this point, no new classes are produced and the procedure terminates. The result accepts exactly the pairs in which the token is moved one or more positions to the right, and it is equivalent to the transducer of Figure 2(b). Composing it with the initial configurations gives the reachable set $\mathsf{n}^*\mathsf{t}\mathsf{n}^*$, which does not intersect the bad configurations, so the protocol is safe.

4.4 Further Techniques

L: armc with cegar is nice, pointer prog verification is a nice application of artmc

Acceleration is one of several techniques for computing transitive closures or reachable sets in regular model checking. Related constructions iterate transducers by identifying equivalent states with bisimulation-like relations [DLS01], and the quotienting technique above has been made more efficient using simulations [AJNd03]. *Widening* and *extrapolation* techniques compute a few iterates, detect a regular pattern of growth, and extrapolate it to a candidate for the limit, which is then checked [Tou01, BLW03, BT02]. *Abstract regular model checking* [BHV04] gives up exactness: it alternates between applying the transducer and abstracting the resulting automata, for example by merging states that accept the same words up to a given length, and refines the abstraction when a spurious counterexample is found. For safety verification, it is enough to find a regular set that contains the initial configurations, is closed under the transducer, and is disjoint from the bad configurations. Such a *regular inductive invariant* can be found with automata learning, in which a learner proposes candidate automata and a teacher checks the three conditions [HV05, NJ13, CHLR17].

The framework extends beyond linear words. Tree automata and tree transducers represent configurations of systems with a tree topology or with recursive structure [AJMd02, BT02]. Liveness properties can be handled by working with infinite words [BLW04, BLW05] or through the transitive closure as mentioned above [AJN$^+$04], and regular proofs of liveness have been studied for randomized parameterized systems [LR16]. Surveys of the area can be found in [AJNS04, Abd12, Abd21]. In the next section, we present a technique that replaces regular sets by upward-closed sets, trading generality for efficiency and guaranteed termination.

5 Monotonic Abstraction

Regular model checking is a uniform framework in which a wide class of techniques can be explained. In practice, however, it often needs to be complemented by other techniques. A limiting factor is the computation of the transitive closure of a transducer, which is not guaranteed to terminate and may rely on expensive automata-theoretic constructions. One way to increase efficiency is to over-approximate the behavior of the system. We derive an *abstract* system whose behavior includes that of the original, *concrete*, system, and that is easier to analyze. The price is a loss of precision: the abstract system may contain *false positives*, that is, violations of the safety property that do not exist in the concrete system. The method is nevertheless sound: if the ab-

stract system satisfies a safety property, then so does the concrete system.

In this section, we present an over-approximation, called *monotonic abstraction* [ADBHR07, ABHDR08], that produces a well-structured transition system, so that backward reachability analysis can be applied to the abstract system. We consider the parameterized systems of Section 2, in which the transitions of a process may be constrained by universal or existential global conditions. Global conditions are important in the description of many parameterized systems, such as mutual exclusion protocols, but they are difficult to analyze: parameterized systems with universal conditions can simulate counter machines, so all non-trivial verification problems for them are undecidable. Such systems can be described with transducers as in Section 4, but here we use over-approximation instead.

5.1 Example

We introduce the method through a protocol that implements mutual exclusion among an arbitrary number of processes. Each process, depicted in Figure 5, has four local states: idle (i), requesting (r), waiting (w), and critical (c). Initially, all processes are idle. When a process becomes interested in the critical section, it declares its interest by moving to the requesting state. This is described by the universal transition t_1, which is allowed only if all other processes, whether in the left or the right context, are idle or requesting; this is indicated by the index LR of the quantifier. From the requesting state, a process may move to the waiting state by the local transition t_2, which does not depend on the other processes. Any number of processes may move from idle to requesting. However, once the first process has moved to the waiting state, it closes the door on the processes that are still idle: they cannot leave the idle state until no process is waiting or critical.

Among the processes that have declared their interest, the leftmost process has the highest priority. This is described by the universal transition t_4, by which a process may move from the waiting state to the critical section only if all processes in its left context are idle; the index L stands for left. If there is a process in its left context that is requesting, waiting, or critical, the process loops in the waiting state by the existential transition t_3. When the process leaves the critical section, it returns to the requesting state by the local transition t_5. From the requesting state, it either tries to reach the critical section again, or becomes idle by the local transition t_6.

5.2 Model

In the notation of Section 2, the protocol is the parameterized system with local states $Q = \{\mathsf{i}, \mathsf{r}, \mathsf{w}, \mathsf{c}\}$ and the local rules $t_2 = \mathsf{r} \to \mathsf{w}$, $t_5 = \mathsf{c} \to \mathsf{r}$, and $t_6 = \mathsf{r} \to \mathsf{i}$, together with the global rules

$$t_1 = \textbf{if } \forall j \neq i : \{\mathsf{i}, \mathsf{r}\} \textbf{ then } \mathsf{i} \to \mathsf{r},$$
$$t_3 = \textbf{if } \exists j < i : \{\mathsf{r}, \mathsf{w}, \mathsf{c}\} \textbf{ then } \mathsf{w} \to \mathsf{w},$$
$$t_4 = \textbf{if } \forall j < i : \{\mathsf{i}\} \textbf{ then } \mathsf{w} \to \mathsf{c}.$$

L: introduce somewhere the indexed quantifiers used in the figure

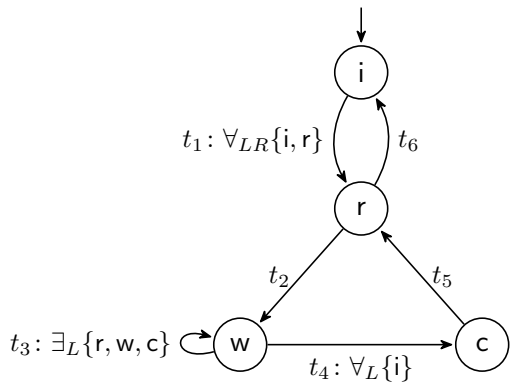


Figure 5: One process in a mutual exclusion protocol with a linear topology. The labels give the global conditions of the transitions t_1, t_3, and t_4; the other transitions are local.

A configuration is a word over Q. For instance, the configuration iwcwr describes an instance with five processes that are idle, waiting, critical, waiting, and requesting, in that order. The transition relation $\to$ depends on the type of the transition rule. The local rule t_2 induces, for instance, the transition wircr $\to$ wiwcr, in which the active process changes its state from requesting to waiting. The existential rule t_3 induces the transition riwcr $\to$ riwcr, since the waiting process has a requesting process in its left context. The same transition is not enabled from iiwcr, since no process in the left context of the waiting process is requesting, waiting, or critical. The universal rule t_4 induces the transition iiwwr $\to$ iicwr, since all processes in the left context of the active process are idle. From ciwwr, neither waiting process can perform t_4, since for each of them, some process in its left context is not idle.

The initial configurations are those in which all processes are idle, such as ii and iiiii. The bad configurations are those in which at least two processes are critical, such as crc and icrcwc. Proving the safety property amounts to showing that no bad configuration is reachable from an initial configuration.

5.3 Orderings and Backward Reachability

To analyze the protocol, we use an ordering on configurations that compares instances of different sizes, together with the theory of well-structured transition systems [AČJT96, AČJT00, FS01], which is presented in Chapter "Well-Structured Systems". We recall here what we need.

A *quasi-ordering* on a set C is a reflexive and transitive relation $\sqsubseteq \subseteq C \times C$. It is a *well-quasi-ordering* (WQO) if every infinite sequence $c_0, c_1, c_2, \ldots$ of elements of C contains indices $i < j$ with $c_i \sqsubseteq c_j$. For words $u, v \in Q^*$, we write $u \preceq v$ if u is a *subword* of v, that is, if $u = v[i_1] \cdots v[i_k]$ for some positions $i_1 < \cdots < i_k$. For example, wc $\preceq$ rwicw. By Higman's lemma [Hig52], the subword ordering is a WQO.

For a set $X \subseteq C$, we define its *upward closure* $\uparrow X = \{c \mid c' \sqsubseteq c \text{ for some } c' \in X\}$, and we say that X is *upward-closed* if $X = \uparrow X$. The *generator* $gen(U)$ of an upward-closed set U is its

set of minimal elements. If $\sqsubseteq$ is a WQO, then $gen(U)$ is finite and $U = {\uparrow}gen(U)$, so upward-closed sets can be represented by finite sets. Moreover, every increasing sequence $U_0 \subseteq U_1 \subseteq \cdots$ of upward-closed sets stabilizes after finitely many steps. In our example, the set of bad configurations is the upward-closed set ${\uparrow}\{\mathsf{cc}\}$ of configurations that contain cc as a subword.

L: as clear from the definition ...

A transition relation $\rightarrow$ on C is *monotonic* with respect to $\sqsubseteq$ if, whenever $c_1 \rightarrow c_2$ and $c_1 \sqsubseteq c_3$, there is a configuration c_4 with $c_3 \rightarrow c_4$ and $c_2 \sqsubseteq c_4$. A *well-structured transition system* is a transition system with a WQO on its configurations with respect to which its transition relation is monotonic. If $\rightarrow$ is monotonic and U is upward-closed, then $Pre(U)$ is upward-closed.

L: not hard to show

Suppose that *Bad* is upward-closed. Procedure 2 computes a finite set U with ${\uparrow}U = Pre^*(Bad)$, the set of configurations from which *Bad* is reachable. It starts from the generator of *Bad* and adds the minimal predecessors of ${\uparrow}U$ until no new configuration is found. A configuration c is discarded if $c' \sqsubseteq c$ for some c' already in U; we say that c is *subsumed* by c', since ${\uparrow}c \subseteq {\uparrow}c'$. The system is safe if and only if ${\uparrow}U \cap Init = \emptyset$.

Procedure 2 Backward reachability analysis

Require: A well-structured transition system, a set *Init*, and the generator G of an upward-closed set *Bad*
Ensure: Whether *Bad* is reachable from *Init*

1: $U \leftarrow G$
2: **repeat**
3: $\quad N \leftarrow \{c \in gen(Pre({\uparrow}U)) \mid c' \not\sqsubseteq c \text{ for all } c' \in U\}$
4: $\quad U \leftarrow U \cup N$
5: **until** $N = \emptyset$
6: **return** whether ${\uparrow}U \cap Init \neq \emptyset$

Theorem 1 ([AČJT96, AČJT00]) *Procedure 2 terminates and returns the correct answer for every transition system $(C, \rightarrow)$ with an ordering $\sqsubseteq$ that satisfies the following conditions:*

1. *$\rightarrow$ is monotonic with respect to $\sqsubseteq$;*
2. *$\sqsubseteq$ is a WQO;*
3. *for every $c \in C$, the finite set $gen(Pre({\uparrow}\{c\}))$ is computable;*
4. *$\sqsubseteq$ is decidable;*
5. *for every finite set $B \subseteq C$, the emptiness of ${\uparrow}B \cap Init$ is decidable.*

Conditions 1 and 2 ensure that each set computed by the procedure is upward-closed and that the sequence of sets stabilizes; conditions 3 to 5 make each step effective. Petri nets, broadcast protocols, and lossy channel systems satisfy these conditions [AČJT96, EFM99]. Our protocol satisfies conditions 2 to 5 with respect to the subword ordering, but not condition 1, as we show next.

5.4 Over-Approximation

We now define an abstract system that over-approximates the concrete one and that satisfies the conditions of Theorem 1 with respect to the subword ordering $\preceq$. The main task is to obtain a

transition relation that is monotonic with respect to $\preceq$. We first explain why local and existential transitions are already monotonic, and why universal transitions are not.

Consider the transition irc $\rightarrow$ iwc induced by the local rule t_2, and the larger configuration iwircr. The larger configuration can perform the same local transition and reach iwiwcr, which is larger than iwc. In general, the active process of the smaller configuration also exists in the larger configuration, and since a local transition neither checks nor changes the states of the other processes, the larger configuration can perform the transition while preserving the ordering.

Consider next the transition riwcr $\rightarrow$ riwcr induced by the existential rule t_3. The configuration consists of the active process in state w, the left context ri, and the right context cr, and the left context contains a *witness*, the requesting process, that enables the transition. In the larger configuration iriwcrc, the left and right contexts of the waiting process are larger than in the smaller configuration, and in particular the left context still contains the witness. Hence the larger configuration can perform the same transition.

Universal transitions, in contrast, are not monotonic. Consider the transition iiwwr $\rightarrow$ iicwr induced by the universal rule t_4, which is enabled since the left context of the active process is idle. The larger configuration irirwwr cannot perform the transition, since the left context of the active process now contains requesting processes, which violate the condition.

Since local and existential transitions are monotonic, they need not be approximated. For universal transitions, we define an abstract transition relation $\rightsquigarrow$ as follows. An abstract transition first deletes all processes that violate the condition of the universal rule, and then performs the concrete transition. For instance, we have

$$\mathsf{irirwwr} \rightsquigarrow \mathsf{iicwr},$$

since we can first delete the two requesting processes in the left context, obtaining iiwwr, and then perform the concrete transition t_4. For local and existential rules, $\rightsquigarrow$ coincides with $\rightarrow$. Every concrete transition is also an abstract transition, so the abstract system over-approximates the concrete one, and the abstract transition relation is monotonic with respect to $\preceq$.

5.5 Backward Reachability Analysis

The abstract system satisfies the conditions of Theorem 1. The subword ordering is a WQO, and it is decidable, since checking whether one word is a subword of another is easy. The abstract transition relation is monotonic, as explained above. For a finite set B of configurations and a regular set *Init* of initial configurations, the emptiness of $\uparrow B \cap \mathit{Init}$ amounts to the emptiness of the intersection of two regular languages. It remains to show how to compute, for a configuration c and a rule t, the generator $\mathit{Pre}(t)(c)$ of the set of configurations from which $\uparrow c$ can be reached by one abstract transition induced by t. A formal definition can be found in [ADBHR07]; here we illustrate the computation for the different types of rules of Figure 5.

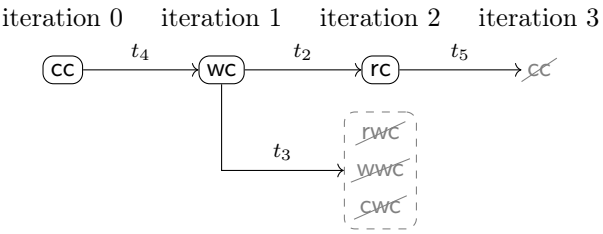


Figure 6: Backward reachability analysis for the protocol of Figure 5. The framed configurations form the generator of the set of configurations from which a bad configuration is reachable in the abstract system. The struck-out configurations are subsumed by configurations found earlier.

For a local rule, we run the transition backwards on the active process. For instance, for the local rule t_5, we have $Pre(t_5)(\mathsf{irw}) = \{\mathsf{icw}\}$. Strictly speaking, the predecessors also include configurations such as ircw, but these are subsumed by irw itself, and we omit them for simplicity.

For an existential rule, there are two cases, depending on whether the configuration contains a witness. For the rule t_3, we have $Pre(t_3)(\mathsf{rwc}) = \{\mathsf{rwc}\}$, since the left context of the waiting process contains a requesting process. In contrast,

$$Pre(t_3)(\mathsf{iwc}) = \{\mathsf{riwc}, \mathsf{irwc}, \mathsf{wiwc}, \mathsf{iwwc}, \mathsf{ciwc}, \mathsf{icwc}\},$$

since there is no witness in the left context of the active process, and we add a witness explicitly, in each possible state and at each possible position in the left context. The new configurations have four processes, while the original configuration has three. The configurations generated by the backward analysis may thus grow, and there is no a priori bound on their size.

For a universal rule, we check whether some process in the configuration violates the condition. For the rule t_4, we have $Pre(t_4)(\mathsf{iricw}) = \emptyset$, since the left context of the critical process, which would be the active process, contains a requesting process. Such a process would have been deleted by the abstract transition. In contrast, $Pre(t_4)(\mathsf{iicw}) = \{\mathsf{iiww}\}$, since all processes in the left context of the active process are idle.

We can now run the backward reachability analysis of Section 5.3 on the abstract system (Figure 6). The analysis starts from the generator cc of the bad configurations. The only rule that produces a critical process is t_4. Of the two critical processes in cc, only the left one can perform t_4 backwards, since the left context of the right one contains a process that is not idle, which would have been deleted by the abstract transition. This gives $Pre(t_4)(\mathsf{cc}) = \{\mathsf{wc}\}$. From wc, two rules are enabled backwards, both for the waiting process: the local rule t_2 gives $Pre(t_2)(\mathsf{wc}) = \{\mathsf{rc}\}$, and the existential rule t_3 gives $Pre(t_3)(\mathsf{wc}) = \{\mathsf{rwc}, \mathsf{wwc}, \mathsf{cwc}\}$. All three configurations produced by t_3 are subsumed by wc. From rc, the only rule enabled backwards is the local rule t_5, which gives cc, which is subsumed. The rule t_1 cannot be applied backwards, since the critical process violates its condition and would have been deleted. At this point, no new configurations are produced, and

the analysis terminates.

The set of configurations from which a bad configuration is reachable in the abstract system is therefore $\uparrow\{\mathsf{cc}, \mathsf{wc}, \mathsf{rc}\}$. It contains no initial configuration, since the initial configurations contain only idle processes. Hence no bad configuration is reachable in the abstract system, and, since $\rightarrow \subseteq \rightsquigarrow$, none is reachable in the concrete system either. In this example, the abstraction does not lose any precision with respect to the property: the configurations that are reachable in the concrete system are exactly those that contain none of cc, wc, and rc as a subword. Intuitively, a process enters the critical section only when all processes to its left are idle, and these processes cannot leave the idle state while it is critical. We return to this observation in Section 6.

L: technically, subset of this set, as it is an overapproximation (?)

5.6 Extensions

Monotonic abstraction has been extended in several directions. In many protocols, such as Burns' and Szymanski's algorithms, a global condition is not checked atomically: the process inspects the other processes one at a time, and the other processes may move during the inspection. Such non-atomic global conditions can be handled by a refinement of the abstraction [ABHDR08]. Processes may also have numerical variables, for instance clocks or tickets, whose values are compared using gap-order constraints [ADR09]. The method has been applied to tree topologies, to broadcast communication and binary rendezvous, and to German's directory-based cache coherence protocol [ADBHR07]. Since the abstraction may introduce false positives, it has been combined with a counterexample-guided refinement loop, which restricts the deletions allowed in the abstract transitions when a spurious counterexample is found [ACD+10]. Monotonic abstraction is also related to the verification of programs running under weak memory models, where the buffers of the processes raise similar questions of monotonicity, see Section 7 and Chapter "Weak Memory Models".

6 View Abstraction

Monotonic abstraction computes the configurations from which a bad configuration is reachable, starting from the bad configurations and going backwards. In this section, we present *view abstraction* [AHH13, AHH16], a forward analysis that exploits a *small model property* of many parameterized systems: the behavior of large instances is already visible in small ones. A configuration of a large instance can often be seen as a combination of patterns that already occur in configurations of small instances, and if a bad pattern exists, it can often be found with only a few processes. The method therefore inspects only small instances of the system, and abstracts the configurations it finds into *views*, which describe the system from the perspective of a fixed number of processes. The views are then recombined to obtain an over-approximation of the reachable configurations of all sizes. The number of processes that need to be inspected is not fixed in advance: the method discovers a *cutoff point* during the analysis, beyond which the search need not continue. In this

sense, view abstraction is a *dynamic cutoff* method, in contrast to the static cutoff theorems discussed in Section 7. Through the theory of Section 5.3, the method is complete for a large class of well-structured systems.

6.1 Model

We consider parameterized systems with a linear topology as defined in Section 2, and we assume that global conditions are checked atomically. For instance, the rule t_4 of Figure 5 is written **if** $\forall j < i : \{\mathsf{i}\}$ **then** $\mathsf{w} \to \mathsf{c}$. We assume that $Bad = \uparrow B_{\min}$ is the upward closure of a finite set $B_{\min}$ of minimal bad configurations with respect to the subword ordering. We write $\mathcal{R}_k$ for the set of configurations of size at most k that are reachable from initial configurations of size at most k. Since the transitions do not change the size of a configuration, $\mathcal{R}_k$ is finite and can be computed by explicit-state exploration.

6.2 Views

For a natural number k, a *view* of size k of a configuration is a subword of the configuration of length k. A view retains the states of k processes, in the order in which they occur in the configuration, and ignores the other processes. We write $\mathcal{V}_k$ for the set of words of length at most k over Q, which are the possible views of size up to k. The *abstraction function* α_k maps a configuration c to the set of its views of size up to k, that is, to the set of words $v \in \mathcal{V}_k$ with $v \preceq c$. For a set X of configurations, $\alpha_k(X)$ is the union of the sets $\alpha_k(c)$ with $c \in X$. The *concretization function* γ_k maps a set V of views to the set of all configurations that can be built from views in V:

$$\gamma_k(V) = \{c \mid \alpha_k(c) \subseteq V\}.$$

The two functions form a Galois connection, so $X \subseteq \gamma_k(\alpha_k(X))$ for every set X of configurations.

The concretization is an over-approximation. Consider the set $X = \{\mathsf{irw}, \mathsf{rwc}, \mathsf{iwc}\}$. Its views of size up to two are the four single states together with ir, iw, rw, rc, wc, and ic. The concretization of these views contains the three configurations of X, but also, for instance, the configuration irc, all of whose views of size two belong to the set, and the configuration irwc of size four. The views work together to represent a set of configurations: a configuration belongs to the concretization only if all its views do. Larger views impose more constraints, and the precision increases with k: for every set X of configurations, we have $\gamma_{k+1}(\alpha_{k+1}(X)) \subseteq \gamma_k(\alpha_k(X))$.

The *abstract post-image* of a set V of views is defined as usual by $Apost_k(V) = \alpha_k(Post(\gamma_k(V)))$. If a set V of views contains the views of the initial configurations and satisfies $Apost_k(V) \subseteq V$, then $\gamma_k(V)$ contains all reachable configurations, that is, $\mathcal{R} \subseteq \gamma_k(V)$.

6.3 The Procedure

The verification procedure (Procedure 3) consists of two nested loops. The outer loop increases the precision k. For a given k,

Procedure 3 Verification by view abstraction

1: **for** $k = 1, 2, 3, \ldots$ **do**
2: **if** $\mathcal{R}_k \cap Bad \neq \emptyset$ **then**
3: **return** unsafe
4: **end if**
5: $V \leftarrow$ the least set of views that contains $\alpha_k(Init)$ and satisfies $Apost_k(V) \subseteq V$
6: **if** $\gamma_k(V) \cap Bad = \emptyset$ **then**
7: **return** safe
8: **end if**
9: **end for**

the procedure first computes the reachable configurations of size up to k exactly. If one of them is bad, the system is unsafe, and the run that reaches it is a counterexample. Otherwise, the procedure computes a fixpoint of the abstract post-image, starting from the views of the initial configurations. If the concretization of the fixpoint contains no bad configuration, the system is safe for all sizes. Otherwise, the result is inconclusive, since the bad configuration may be an artifact of the abstraction, and the procedure increases k. The value of k for which the procedure terminates is a dynamic cutoff.

The fixpoint computation cannot be performed directly, since $\gamma_k(V)$ is infinite in general. The key observation is that it suffices to consider the configurations of size at most $k+1$ in $\gamma_k(V)$:

$$\alpha_k(Post(\gamma_k(V))) \cup V = \alpha_k(Post(\gamma_k^{k+1}(V))) \cup V,$$

where $\gamma_k^{k+1}(V)$ is the finite set of configurations of size at most $k+1$ in $\gamma_k(V)$. The reason is that a transition changes the state of one process, so a new view must contain the active process. For a local or a universal rule, the transition remains enabled if we remove processes other than the active one, since a universal condition only becomes easier to satisfy. For an existential rule, the transition remains enabled if we keep the active process and one witness. In both cases, every new view of size k that is produced from a large configuration is also produced from a configuration of size at most $k+1$ in $\gamma_k(V)$, namely from the view extended with a witness. The test $\gamma_k(V) \cap Bad = \emptyset$ is also finite: it amounts to checking that no configuration $b \in B_{\min}$ has all its views of size up to k in V. In practice, the fixpoint computation is accelerated by starting from the views of $\mathcal{R}_k$ and $\mathcal{R}_{k+1}$ rather than from the views of *Init*, and in most experiments this initial set is already a fixpoint or close to one [AHH16].

L: maybe remind them that they are obtained by running k threads exactly

Example. We apply the procedure to the protocol of Figure 5. For $k = 1$, the reachable configurations of size one contain no bad configuration, and the fixpoint consists of the four views i, r, w, and c. Its concretization contains all configurations, including cc, so the result is inconclusive. For $k = 2$, the reachable configurations of sizes one and two contain no bad configuration, and their views of size two are all words of length two except cc, rc, and wc. The abstract post-image of these views, computed from

the configurations of size at most three in their concretization, produces no new view, so the fixpoint is reached immediately. Since cc is not among the views, the concretization contains no bad configuration, and the protocol is safe for all sizes, with dynamic cutoff 2. The fixpoint describes exactly the configurations that contain none of cc, rc, and wc as a subword. These are the complement of the set computed by monotonic abstraction in Section 5, which was generated by the same three configurations. The two methods arrive at the same invariant, one going forward from the initial configurations and the other going backward from the bad ones.

6.4 Termination and Completeness

The procedure is sound: when it returns safe, the system is safe, and when it returns unsafe, the counterexample is real. Termination cannot be guaranteed in general, since the problem is undecidable. However, the theory of well-quasi-orderings of Section 5.3 identifies a large class of systems for which the procedure always terminates.

We first establish when the reachable set $\mathcal{R}$ can be represented exactly by views. For a set X of configurations, we define its *downward closure* $\downarrow X$ as the set of configurations c with $c \preceq c'$ for some $c' \in X$, and we say that X is *downward-closed* if $X = \downarrow X$. Suppose that $\mathcal{R}$ is downward-closed. Its complement is then upward-closed, and by Higman's lemma it has a finite generator. Let K be the maximal size of a configuration in this generator. Then $\gamma_K(\alpha_K(\mathcal{R})) = \mathcal{R}$: a configuration outside $\mathcal{R}$ contains an element of the generator as a subword, which is a view of size at most K that does not occur in any reachable configuration. Thus, $\mathcal{R}$ can be represented exactly by views of size at most K.

Secondly, monotonicity guarantees a sufficiently precise representation even when $\mathcal{R}$ is not downward-closed. If the transition relation is monotonic with respect to the subword ordering and *Bad* is upward-closed, then $\downarrow\mathcal{R}$ is closed under transitions, and a bad configuration is reachable if and only if $\downarrow\mathcal{R}$ contains one. Thus, $\downarrow\mathcal{R}$ is an inductive invariant sufficient for proving safety. Since it is downward-closed, it is exactly representable by view abstraction. This covers systems with local and existential transitions, and, after the extensions discussed below, Petri nets, broadcast protocols, and lossy channel systems. The result extends to general well-structured transition systems, provided that the configurations can be measured by a size that has finitely many configurations of each value [AHH16].

Finally, the procedure does indeed compute this representation. For sufficiently large K, view abstraction represents the inductive invariant $\downarrow\mathcal{R}$ exactly, so $\alpha_K(\downarrow\mathcal{R})$ is closed under the abstract post-image. Since the procedure computes the least abstract fixpoint, its result V is contained in $\alpha_K(\downarrow\mathcal{R})$. Conversely, soundness gives $\alpha_K(\mathcal{R}) \subseteq V$. Taking the downward closure introduces no new views, so $\alpha_K(\mathcal{R}) = \alpha_K(\downarrow\mathcal{R})$, and we conclude that $V = \alpha_K(\downarrow\mathcal{R})$. Hence $\gamma_K(V) = \downarrow\mathcal{R}$, so the procedure proves safety by $k = K$ whenever the system is safe. Unsafe systems are eventually detected by checking sufficiently large finite instances.

For these systems, view abstraction provides a forward alternative to the backward reachability analysis of Section 5.3. The protocol of Figure 5 shows that the method can also succeed without monotonicity: its universal transitions are not monotonic, but its reachable set is downward-closed. The generator of its complement is $\{\mathsf{cc}, \mathsf{rc}, \mathsf{wc}\}$, giving $K = 2$.

6.5 Extensions

The method extends to other communication primitives and topologies by adapting the abstraction function, the concretization, and the post-image. For broadcast and rendez-vous transitions, a transition may require the presence of several witnesses, and the configurations used in the abstract post-image must be larger, of size up to $k + m$ when a rendez-vous involves $m + 1$ processes. Shared variables are modeled by a controller process whose state is kept in every view. For multisets, views are submultisets; for rings, they are circular subwords; and for trees, they are subtrees of a given depth [AHH16].

Some systems have no good downward-closed invariant, and their analysis with the views above produces false counterexamples for every k. A typical case is a universal transition that is disabled in all reachable configurations by a process that a view of size k ignores. To handle such systems, *context-sensitive views* record, in addition to the k retained processes, the sets of states of the ignored processes between them. These contexts are taken into account when universal conditions are evaluated, which blocks the spurious transitions [AHH14, AHH16]. A further refinement of the views handles global conditions that are not checked atomically, by recording for each process how far it has progressed in its inspection of the others. With these extensions, the method verified the full, non-atomic version of Szymanski's mutual exclusion protocol, which, to the knowledge of the authors of [AHH16], had not been verified automatically before. In the experiments reported in [AHH16], which include cache coherence protocols, protocols on rings and trees, and Petri nets with inhibitor arcs, the method terminated almost immediately on most examples, which confirms that the relevant patterns already occur in small instances.

L: maybe the lock-free pointers with Fred can be mentioned

7 Other Approaches

The techniques presented in the previous sections are a small part of a large body of work. This section gives an overview of further approaches, with pointers to the literature and to other chapters of this handbook. More detailed surveys can be found in [AST18, BJK+15, Esp14, ZP04].

7.1 Counting Abstraction and Well-Structured Models

When processes are anonymous, a configuration can be represented by counting the number of processes in each local state. This *counting abstraction* is exact, and it turns a system of finite-state processes that communicate through rendez-vous or shared variables into a Petri net, or equivalently a vector addition system. German and Sistla [GS92] considered systems consisting

of a control process and an arbitrary number of identical user processes, communicating by rendez-vous. They gave decision procedures for indexed temporal properties, which run in time polynomial in the size of the process template when all processes are identical, and in doubly exponential time in the presence of a control process. Safety properties reduce to the coverability problem for Petri nets, which is ExpSpace-complete [Rac78, Esp98] and can be solved by the backward reachability analysis of Section 5.3. For liveness properties of the control process, the property is first combined with the control process through a Büchi automaton, and the existence of a fair run that visits accepting states infinitely often is reduced to the reachability problem for Petri nets, which is decidable [GS92].

Broadcast protocols [EFM99] extend Petri nets with transitions in which all processes in a given state react to a message. They are well-structured, so safety properties expressed by upward-closed sets are decidable, while liveness properties become undecidable. Delzanno [Del00] applied counting abstraction with symbolic representations based on linear arithmetic to verify cache coherence protocols for an arbitrary number of caches. These ideas have been extended to systems with pairwise rendez-vous on various topologies [AKR$^+$18], to asynchronous shared-memory systems in which anonymous processes communicate through shared registers [EGM16], to population protocols [AAD$^+$06, EGLM17], and to ad hoc networks, in which processes broadcast to their neighbors in an unknown graph [DSZ10]. A recurring observation is that, when processes are anonymous and any behavior of one process can be replicated by others, the complexity of parameterized verification is often much lower than that of Petri net coverability [Esp14]. The general theory of well-structured transition systems, and the many models to which it applies, are the subject of Chapter "Well-Structured Systems".

7.2 Cutoffs

A *cutoff* for a class of systems and properties is a number k such that a property holds for all instances of a system whenever it holds for all instances of size at most k. A cutoff theorem reduces parameterized verification to finitely many finite-state model checking problems. An early approach of this kind establishes a correspondence between systems of different sizes that preserves an indexed version of CTL* without the next-time operator [BCG89]. Emerson and Namjoshi [EN95] showed that, in rings in which a single token without data is passed between the processes, properties that refer to a fixed number of processes have a small cutoff that is independent of the process template. The result was extended to other topologies by decomposing networks into small fragments [CTTV04], and the existence of cutoffs for token-passing systems has been characterized in terms of the topology and the specification logic [AJKR14].

Emerson and Kahlon [EK00] considered *guarded protocols*, in which a transition is guarded by a *conjunctive* condition, requiring all other processes to be in given states, or by a *disjunctive* condition, requiring at least one of them to be. They estab-

L: maybe say that it is similar to our uni-

lished cutoffs that grow linearly with the number of local states of the template. For disjunctive guards, the argument is that each guard needs only one witness, so a small instance that keeps one process in each local state that occurs in a run can simulate the run. For conjunctive guards, removing processes can only make the guards easier to satisfy, so the projection of a run onto a subset of the processes is a run of a smaller instance; liveness properties require more care, since the projection of a fair run need not be fair. The cutoffs were later sharpened and extended to fairness and deadlock detection [AJK16], and specialized cutoff methods were developed for cache coherence protocols [EK03]. Static cutoffs may be large, and each theorem covers a specific class of systems. Dynamic cutoff methods, such as view abstraction (Section 6) and the method of Kaiser, Kroening, and Wahl [KKW10] for programs with an unbounded number of threads, discover the cutoff during the analysis.

7.3 Abstraction and Network Invariants

A *network invariant* is a process I that abstracts the composition of any number of copies of the template. Kurshan and McMillan [KM89], and independently Wolper and Lovinfosse [WL89], observed that it suffices to check that I abstracts a single copy, and that the composition of I with one more copy is again abstracted by I; verifying the composition of I with a fixed environment then establishes the property for all sizes. The approach was generalized to networks generated by grammars [CGJ97], to invariants given by automata for linear networks [SG99], and to invariants that are computed automatically [LHR97] or established deductively [KPSZ02]. Symmetry reduction exploits the fact that the identity of an individual process is irrelevant, and explores equivalence classes of configurations instead of configurations [ES96, CEFJ96, ID96, GS97]; it applies to each instance separately, but it is the basis of the counting abstraction above.

Several abstractions summarize the processes by a finite amount of information. $(0, 1, \infty)$-*counter abstraction* [PXZ02] records, for each local state, whether zero, one, or many processes occupy it, and adds fairness conditions derived from the concrete system so that liveness properties can be proved. *Environment abstraction* [CTV06, CTV08] describes the system from the point of view of one reference process, by predicates that relate the other processes to it. *Indexed predicates* [LB07] lift predicate abstraction to quantified invariants. For cache coherence protocols, compositional model checking [McM01] and the CMP method [CMP04] keep a few processes concrete and replace the others by a single abstract process, whose behavior is constrained by noninterference lemmas; many of these lemmas can be derived from the message flows of the protocol [TT08]. For programs with an unbounded number of identical threads, *thread-modular* reasoning verifies one thread against an assumption on the others [FQ03], and its precision can be increased by reasoning about several threads at a time [HMP17, FKP15].

L: similar to view abstraction refinement

7.4 Deductive and SMT-Based Methods

Deductive methods prove that a quantified formula is an inductive invariant, that is, that it holds initially and is preserved by every transition, for an arbitrary number of processes. The approach goes back to the proof method of Owicki and Gries [OG76], and is supported by languages and tools such as TLA+ [Lam94, Lam02, CDL+12], which has been applied to industrial protocols [NRZ+15]. The main difficulty is to find the invariant. The *invisible invariants* method [PRZ01, APR+01] computes the reachable configurations of a small instance, projects them onto one or two processes, and generalizes the result to a quantified candidate invariant, which is then checked for all sizes, for a restricted class of systems by a small-model argument.

SMT-based methods automate the search for invariants with satisfiability modulo theories solvers [BSST09]. In *array-based systems* [GR10a, GR10b], the state is a collection of arrays indexed by process identifiers, and backward reachability is carried out on existentially quantified formulas; universal guards are handled by an approximation closely related to monotonic abstraction. The Cubicle model checker [CGK+12] follows the same principles and uses finite instances to guess invariants [CGK+13]. The IC3/PDR algorithm [Bra11, EMB11] has been extended to infer universally quantified invariants [KBI+17, GSM16]. The Ivy tool [PMP+16] restricts verification conditions to a decidable fragment of first-order logic, which yields small counterexamples to induction that the user generalizes interactively, and complex consensus protocols such as Paxos have been verified within this fragment [PLSS17]. The inference of quantified invariants for distributed protocols has also been automated, for instance by generalizing invariants of small instances [MGJ+19] or by searching for quantified separators [KPIA20]. For fault-tolerant distributed algorithms, in which processes wait for messages from a number of other processes that depends on the parameters, *threshold automata* and a short counterexample property reduce parameterized verification to bounded model checking with an SMT solver [JKS+13, KVW17, KLVW17].

7.5 Weak Memory Models

Parameterized verification is also relevant for concurrent programs running under weak memory models, where hardware reordering of memory operations is described by unbounded buffers associated with the threads [OSS09, SSO+10, AMT14]. Synchronization primitives, such as locks and mutual exclusion protocols, are written to be used by any number of threads, and their correctness must be established for all numbers of threads under the memory model of the architecture. For a fixed number of threads, safety verification under Total Store Order (TSO) is decidable by a reduction to lossy channel systems [ABBM10]. A *dual* semantics, in which threads read from load buffers instead of writing to store buffers, makes the system monotonic with respect to a WQO, which yields the first decidability result for parameterized verification under TSO through backward reach-

ability [AABN16, AABN18]. The problem was later shown to be PSPACE-complete [AAR20], and parameterized verification is also PSPACE-complete under the release-acquire semantics of C11 [KGMC22]. For the richer Intel x86 model, which includes non-temporal writes [RMV22, AAB+24], parameterized verification requires *better quasi-orderings* [AAB+26, Mar94], which had earlier been used in verification for timed Petri nets [AN00, Abd10]. Verification under weak memory models is the subject of Chapter "Weak Memory Models".

7.6 Parameterized Quantum Circuits

Parameterized verification has recently been applied to quantum programs, in which the parameter is the number of qubits. Most quantum algorithms, such as Shor's factoring algorithm [Sho97], Grover's search [Gro96], and the quantum Fourier transform, are described by programs that generate a circuit for each input size n, and their correctness is a relational property that relates input states to output states for all n. The framework of [ACH+26] represents infinite families of quantum states by *synchronized weighted tree automata*, in which an n-qubit state is a binary tree of depth n whose leaves carry complex amplitudes. Quantum gates are represented by transducers, which are composed to obtain a transducer for the whole parameterized program, in analogy with the transitive closure of transducers in regular model checking (Section 4). Verification reduces to functional inclusion and equivalence of these automata, which are decidable, and the framework gives the first fully automatic verification of parameterized quantum programs such as the Bernstein-Vazirani algorithm, a ripple-carry adder, and optimized implementations of Grover's search. It builds on automata-based verification of quantum circuits of fixed size [CCL+23b, CCL+23a, ACC+25, CCH+25]. Deductive approaches, based on quantum Hoare logic [Yin11], on deductive verification frameworks [CBB+21], on proof assistants [HRH+21], or on path sums [Amy19], are complementary. Quantum systems are the subject of Chapter "Quantum Systems".

8 Industrial Applications

9 Relations to Other Chapters

10 Conclusion and Open Problems

Parameterized verification asks for guarantees that hold for every number of components, a question that is undecidable in general. The techniques presented in this chapter respond to this difficulty in different ways, but the same few ideas recur across all of them: a finite symbolic representation of infinite sets of configurations, monotonicity with respect to a well-quasi-ordering, and the observation that the behavior of a large instance is often already visible in a small one. The protocol of Section 5 illustrates how these ideas meet: monotonic abstraction, going backwards from the bad configurations, and view abstraction, going forwards from the initial ones, arrive at the same invariant, which is described by three small configurations.

Several problems remain open or only partially solved. Liveness properties under fairness are much less well supported than safety properties, both in theory and in tools, since abstractions that preserve safety often lose the information needed for fairness. Many protocols have invariants that are not downward-closed, and finding good representations for such invariants, as attempted by context-sensitive views, remains a challenge. For weak memory models, parameterized verification is well understood for TSO and some of its extensions, but the more relaxed models of ARM and POWER processors, and the language-level models of C11 and Java, are only partly covered. For distributed protocols, the automatic inference of invariants with alternating quantifiers, which are needed for many consensus protocols, remains difficult. In the quantum setting, the automata-based approach covers parameterized circuits, and its extension to programs with measurement, classical control, and noise is a natural next step.

References

[AAB+24] Parosh Aziz Abdulla, Mohamed Faouzi Atig, Ahmed Bouajjani, K. Narayan Kumar, and Prakash Saivasan. Verification under Intel-x86 with persistency. *Proceedings of the ACM on Programming Languages*, 8(PLDI):1189–1212, 2024.

[AAB+26] Parosh Aziz Abdulla, Mohamed Faouzi Atig, Ahmed Bouajjani, K. Narayan Kumar, and Prakash Saivasan. Parametrised verification of Intel-x86 programs. *Proceedings of the ACM on Programming Languages*, 10(POPL):1094–1122, 2026.

[AABN16] Parosh Aziz Abdulla, Mohamed Faouzi Atig, Ahmed Bouajjani, and Tuan Phong Ngo. The benefits of duality in verifying concurrent programs under TSO. In *Proceedings of CONCUR 2016*, volume 59 of *Leibniz International Proceedings in Informatics*, pages 5:1–5:15. Schloss Dagstuhl, 2016.

[AABN18] Parosh Aziz Abdulla, Mohamed Faouzi Atig, Ahmed Bouajjani, and Tuan Phong Ngo. A load-buffer semantics for total store ordering. *Logical Methods in Computer Science*, 14(1), 2018.

[AAD+06] Dana Angluin, James Aspnes, Zoë Diamadi, Michael J. Fischer, and René Peralta. Computation in networks of passively mobile finite-state sensors. *Distributed Computing*, 18(4):235–253, 2006.

[AAR20] Parosh Aziz Abdulla, Mohamed Faouzi Atig, and Rojin Rezvan. Parameterized verification under TSO is PSPACE-complete. *Proceedings of the ACM on Programming Languages*, 4(POPL):1–29, 2020.

[ABBM10] Mohamed Faouzi Atig, Ahmed Bouajjani, Sebastian Burckhardt, and Madanlal Musuvathi. On the verification problem for weak memory models. In *Proceedings of POPL 2010*, pages 7–18. ACM, 2010.

[Abd10] Parosh Aziz Abdulla. Well (and better) quasi-ordered transition systems. *The Bulletin of Symbolic Logic*, 16(4):457–515, 2010.

[Abd12] Parosh Aziz Abdulla. Regular model checking. *International Journal on Software Tools for Technology Transfer*, 14(2):109–118, 2012.

[Abd21] Parosh Aziz Abdulla. Regular model checking: Evolution and perspectives. In Ernst-Rüdiger Olderog, Bernhard Steffen, and Wang Yi, editors, *Model Checking, Synthesis, and Learning*, volume 13030 of *Lecture Notes in Computer Science*, pages 78–96. Springer, 2021.

[ABHDR08] Parosh Aziz Abdulla, Noomene Ben Henda, Giorgio Delzanno, and Ahmed Rezine. Handling parameterized systems with non-atomic global conditions. In *Proceedings of VMCAI 2008*, volume 4905 of *Lecture Notes in Computer Science*, pages 22–36. Springer, 2008.

[ACC+25] Parosh Aziz Abdulla, Yo-Ga Chen, Yu-Fang Chen, Lukáš Holík, Ondřej Lengál, Jyun-Ao Lin, Fang-Yi Lo, and Wei-Lun Tsai. Verifying quantum circuits with level-synchronized tree automata. *Proceedings of the ACM on Programming Languages*, 9(POPL):923–953, 2025.

[ACD+10] Parosh Aziz Abdulla, Yu-Fang Chen, Giorgio Delzanno, Frédéric Haziza, Chih-Duo Hong, and Ahmed Rezine. Constrained monotonic abstraction: A CEGAR for parameterized verification. In *Proceedings of CONCUR 2010*, volume 6269 of *Lecture Notes in Computer Science*, pages 86–101. Springer, 2010.

[ACH+26] Parosh Aziz Abdulla, Yu-Fang Chen, Michal Hečko, Lukáš Holík, Ondřej Lengál, Jyun-Ao Lin, and Ramanathan S. Thinniyam. Parameterized verification of quantum circuits. *Proceedings of the ACM on Programming Languages*, 10(POPL):2021–2050, 2026.

[AČJT96] Parosh Aziz Abdulla, Kārlis Čerāns, Bengt Jonsson, and Yih-Kuen Tsay. General decidability theorems for infinite-state systems. In *Proceedings of LICS 1996*, pages 313–321. IEEE Computer Society, 1996.

[AČJT00] Parosh Aziz Abdulla, Kārlis Čerāns, Bengt Jonsson, and Yih-Kuen Tsay. Algorithmic analysis of programs with well quasi-ordered domains. *Information and Computation*, 160(1–2):109–127, 2000.

[ADBHR07] Parosh Aziz Abdulla, Giorgio Delzanno, Noomene Ben Henda, and Ahmed Rezine. Regular model checking without transducers (on efficient verification of parameterized systems). In *Proceedings of TACAS 2007*, volume 4424 of *Lecture Notes in Computer Science*, pages 721–736. Springer, 2007.

[ADR09] Parosh Aziz Abdulla, Giorgio Delzanno, and Ahmed Rezine. Approximated parameterized verification of infinite-state processes with global conditions. *Formal Methods in System Design*, 34(2):126–156, 2009.

[AHH13] Parosh Aziz Abdulla, Frédéric Haziza, and Lukáš Holík. All for the price of few: Parameterized verification through view abstraction. In *Proceedings of VMCAI 2013*, volume 7737 of *Lecture Notes in Computer Science*, pages 476–495. Springer, 2013.

[AHH14] Parosh Aziz Abdulla, Frédéric Haziza, and Lukáš Holík. Block me if you can! Context-sensitive parameterized verification. In *Proceedings of SAS 2014*, volume 8723 of

Lecture Notes in Computer Science, pages 1–17. Springer, 2014.

[AHH16] Parosh Aziz Abdulla, Frédéric Haziza, and Lukáš Holík. Parameterized verification through view abstraction. *International Journal on Software Tools for Technology Transfer*, 18(5):495–516, 2016.

[AJK16] Simon Außerlechner, Swen Jacobs, and Ayrat Khalimov. Tight cutoffs for guarded protocols with fairness. In *Proceedings of VMCAI 2016*, volume 9583 of *Lecture Notes in Computer Science*, pages 476–494. Springer, 2016.

[AJKR14] Benjamin Aminof, Swen Jacobs, Ayrat Khalimov, and Sasha Rubin. Parameterized model checking of token-passing systems. In *Proceedings of VMCAI 2014*, volume 8318 of *Lecture Notes in Computer Science*, pages 262–281. Springer, 2014.

[AJMd02] Parosh Aziz Abdulla, Bengt Jonsson, Pritha Mahata, and Julien d'Orso. Regular tree model checking. In *Proceedings of CAV 2002*, volume 2404 of *Lecture Notes in Computer Science*, pages 555–568. Springer, 2002.

[AJN+04] Parosh Aziz Abdulla, Bengt Jonsson, Marcus Nilsson, Julien d'Orso, and Mayank Saksena. Regular model checking for LTL(MSO). In *Proceedings of CAV 2004*, volume 3114 of *Lecture Notes in Computer Science*, pages 348–360. Springer, 2004.

[AJNd02] Parosh Aziz Abdulla, Bengt Jonsson, Marcus Nilsson, and Julien d'Orso. Regular model checking made simple and efficient. In *Proceedings of CONCUR 2002*, volume 2421 of *Lecture Notes in Computer Science*, pages 116–131. Springer, 2002.

[AJNd03] Parosh Aziz Abdulla, Bengt Jonsson, Marcus Nilsson, and Julien d'Orso. Algorithmic improvements in regular model checking. In *Proceedings of CAV 2003*, volume 2725 of *Lecture Notes in Computer Science*, pages 236–248. Springer, 2003.

[AJNS04] Parosh Aziz Abdulla, Bengt Jonsson, Marcus Nilsson, and Mayank Saksena. A survey of regular model checking. In *Proceedings of CONCUR 2004*, volume 3170 of *Lecture Notes in Computer Science*, pages 35–48. Springer, 2004.

[AK86] Krzysztof R. Apt and Dexter C. Kozen. Limits for automatic verification of finite-state concurrent systems. *Information Processing Letters*, 22(6):307–309, 1986.

[AKR+18] Benjamin Aminof, Tomer Kotek, Sasha Rubin, Francesco Spegni, and Helmut Veith. Parameterized model checking of rendezvous systems. *Distributed Computing*, 31(3):187–222, 2018.

[AMT14] Jade Alglave, Luc Maranget, and Michael Tautschnig. Herding cats: Modelling, simulation, testing, and data mining for weak memory. *ACM Transactions on Programming Languages and Systems*, 36(2):1–74, 2014.

[Amy19] Matthew Amy. Towards large-scale functional verification of universal quantum circuits. *Electronic Proceedings in Theoretical Computer Science*, 287:1–21, 2019.

[AN00] Parosh Aziz Abdulla and Aletta Nylén. Better is better than well: On efficient verification of infinite-state systems. In *Proceedings of LICS 2000*, pages 132–140. IEEE Computer Society, 2000.

[APR+01] Tamarah Arons, Amir Pnueli, Sitvanit Ruah, Ying Xu, and Lenore Zuck. Parameterized verification with automatically computed inductive assertions. In *Proceedings of CAV 2001*, volume 2102 of *Lecture Notes in Computer Science*, pages 221–234. Springer, 2001.

[AST18] Parosh Aziz Abdulla, A. Prasad Sistla, and Muralidhar Talupur. Model checking parameterized systems. In Edmund M. Clarke, Thomas A. Henzinger, Helmut Veith, and Roderick Bloem, editors, *Handbook of Model Checking*, pages 685–725. Springer, 2018.

[BCG89] Michael C. Browne, Edmund M. Clarke, and Orna Grumberg. Reasoning about networks with many identical finite state processes. *Information and Computation*, 81(1):13–31, 1989.

[BHV04] Ahmed Bouajjani, Peter Habermehl, and Tomáš Vojnar. Abstract regular model checking. In *Proceedings of CAV 2004*, volume 3114 of *Lecture Notes in Computer Science*, pages 372–386. Springer, 2004.

[BJK+15] Roderick Bloem, Swen Jacobs, Ayrat Khalimov, Igor Konnov, Sasha Rubin, Helmut Veith, and Josef Widder. *Decidability of Parameterized Verification*. Synthesis Lectures on Distributed Computing Theory. Morgan & Claypool, 2015.

[BJNT00] Ahmed Bouajjani, Bengt Jonsson, Marcus Nilsson, and Tayssir Touili. Regular model checking. In *Proceedings of CAV 2000*, volume 1855 of *Lecture Notes in Computer Science*, pages 403–418. Springer, 2000.

[BLW03] Bernard Boigelot, Axel Legay, and Pierre Wolper. Iterating transducers in the large. In *Proceedings of CAV 2003*, volume 2725 of *Lecture Notes in Computer Science*, pages 223–235. Springer, 2003.

[BLW04] Bernard Boigelot, Axel Legay, and Pierre Wolper. Omega-regular model checking. In *Proceedings of TACAS 2004*, volume 2988 of *Lecture Notes in Computer Science*, pages 561–575. Springer, 2004.

[BLW05] Ahmed Bouajjani, Axel Legay, and Pierre Wolper. Handling liveness properties in (ω-)regular model checking. *Electronic Notes in Theoretical Computer Science*, 138(3):101–115, 2005.

[Bra11] Aaron R. Bradley. SAT-based model checking without unrolling. In *Proceedings of VMCAI 2011*, volume 6538 of *Lecture Notes in Computer Science*, pages 70–87. Springer, 2011.

[BSST09] Clark Barrett, Roberto Sebastiani, Sanjit A. Seshia, and Cesare Tinelli. Satisfiability modulo theories. In *Handbook of Satisfiability*, volume 185 of *Frontiers in Artificial Intelligence and Applications*, pages 825–885. IOS Press, 2009.

[BT02] Ahmed Bouajjani and Tayssir Touili. Extrapolating tree transformations. In *Proceedings of CAV 2002*, volume 2404 of *Lecture Notes in Computer Science*, pages 539–554. Springer, 2002.

[CBB+21] Christophe Chareton, Sébastien Bardin, François Bobot, Valentin Perrelle, and Benoît Valiron. An automated deductive verification framework for circuit-building quantum programs. In *Proceedings of ESOP 2021*, volume 12648 of *Lecture Notes in Computer Science*, pages 148–177. Springer, 2021.

[CCH+25] Yu-Fang Chen, Kai-Min Chung, Min-Hsiu Hsieh, Wei-Jia Huang, Ondřej Lengál, Jyun-Ao Lin, and Wei-Lun Tsai. AutoQ 2.0: From verification of quantum circuits to verification of quantum programs. In *Proceedings of TACAS 2025*, Lecture Notes in Computer Science, pages 87–108. Springer, 2025.

[CCL+23a] Yu-Fang Chen, Kai-Min Chung, Ondřej Lengál, Jyun-Ao Lin, and Wei-Lun Tsai. AutoQ: An automata-based quantum circuit verifier. In *Proceedings of CAV 2023*, volume 13966 of *Lecture Notes in Computer Science*, pages 139–153. Springer, 2023.

[CCL+23b] Yu-Fang Chen, Kai-Min Chung, Ondřej Lengál, Jyun-Ao Lin, Wei-Lun Tsai, and Di-De Yen. An automata-based framework for verification and bug hunting in quantum circuits. *Proceedings of the ACM on Programming Languages*, 7(PLDI):1218–1243, 2023.

[CDL+12] Denis Cousineau, Damien Doligez, Leslie Lamport, Stephan Merz, Daniel Ricketts, and Hernán Vanzetto. TLA+ proofs. In *Proceedings of FM 2012*, volume 7436 of *Lecture Notes in Computer Science*, pages 147–154. Springer, 2012.

[CEFJ96] Edmund M. Clarke, Reinhard Enders, Thomas Filkorn, and Somesh Jha. Exploiting symmetry in temporal logic model checking. *Formal Methods in System Design*, 9(1–2):77–104, 1996.

[CGJ97] Edmund M. Clarke, Orna Grumberg, and Somesh Jha. Verifying parameterized networks. *ACM Transactions on Programming Languages and Systems*, 19(5):726–750, 1997.

[CGK+12] Sylvain Conchon, Amit Goel, Sava Krstić, Alain Mebsout, and Fatiha Zaïdi. Cubicle: A parallel SMT-based model checker for parameterized systems. In *Proceedings of CAV 2012*, volume 7358 of *Lecture Notes in Computer Science*, pages 718–724. Springer, 2012.

[CGK+13] Sylvain Conchon, Amit Goel, Sava Krstić, Alain Mebsout, and Fatiha Zaïdi. Invariants for finite instances and beyond. In *Proceedings of FMCAD 2013*, pages 61–68. IEEE, 2013.

[CHLR17] Yu-Fang Chen, Chih-Duo Hong, Anthony W. Lin, and Philipp Rümmer. Learning to prove safety over parameterised concurrent systems. In *Proceedings of FMCAD 2017*, pages 76–83. IEEE, 2017.

[CMP04] Ching-Tsun Chou, Phanindra K. Mannava, and Seungjoon Park. A simple method for parameterized verification of cache coherence protocols. In *Proceedings of FMCAD 2004*, volume 3312 of *Lecture Notes in Computer Science*, pages 382–398. Springer, 2004.

[CTTV04] Edmund M. Clarke, Muralidhar Talupur, Tayssir Touili, and Helmut Veith. Verification by network decomposition. In *Proceedings of CONCUR 2004*, volume 3170 of *Lecture Notes in Computer Science*, pages 276–291. Springer, 2004.

[CTV06] Edmund M. Clarke, Muralidhar Talupur, and Helmut Veith. Environment abstraction for parameterized verification. In *Proceedings of VMCAI 2006*, volume 3855 of *Lecture Notes in Computer Science*, pages 126–141. Springer, 2006.

[CTV08] Edmund M. Clarke, Muralidhar Talupur, and Helmut Veith. Proving Ptolemy right: The environment abstraction framework for model checking concurrent systems. In *Proceedings of TACAS 2008*, volume 4963 of *Lecture Notes in Computer Science*, pages 33–47. Springer, 2008.

[Del00] Giorgio Delzanno. Automatic verification of parameterized cache coherence protocols. In *Proceedings of CAV 2000*, volume 1855 of *Lecture Notes in Computer Science*, pages 53–68. Springer, 2000.

[DLS01] Dennis Dams, Yassine Lakhnech, and Martin Steffen. Iterating transducers. In *Proceedings of CAV 2001*, volume 2102 of *Lecture Notes in Computer Science*, pages 286–297. Springer, 2001.

[DSZ10] Giorgio Delzanno, Arnaud Sangnier, and Gianluigi Zavattaro. Parameterized verification of ad hoc networks. In *Proceedings of CONCUR 2010*, volume 6269 of *Lecture Notes in Computer Science*, pages 313–327. Springer, 2010.

[EFM99] Javier Esparza, Alain Finkel, and Richard Mayr. On the verification of broadcast protocols. In *Proceedings of LICS 1999*, pages 352–359. IEEE Computer Society, 1999.

[EGLM17] Javier Esparza, Pierre Ganty, Jérôme Leroux, and Rupak Majumdar. Verification of population protocols. *Acta Informatica*, 54(2):191–215, 2017.

[EGM16] Javier Esparza, Pierre Ganty, and Rupak Majumdar. Parameterized verification of asynchronous shared-memory systems. *Journal of the ACM*, 63(1):1–48, 2016.

[EK00] E. Allen Emerson and Vineet Kahlon. Reducing model checking of the many to the few. In *Proceedings of CADE 2000*, volume 1831 of *Lecture Notes in Artificial Intelligence*, pages 236–254. Springer, 2000.

[EK03] E. Allen Emerson and Vineet Kahlon. Exact and efficient verification of parameterized cache coherence protocols. In *Proceedings of CHARME 2003*, volume 2860 of *Lecture Notes in Computer Science*, pages 247–262. Springer, 2003.

[EMB11] Niklas Eén, Alan Mishchenko, and Robert K. Brayton. Efficient implementation of property directed reachability. In *Proceedings of FMCAD 2011*, pages 125–134. FMCAD Inc., 2011.

[EN95] E. Allen Emerson and Kedar S. Namjoshi. Reasoning about rings. In *Proceedings of POPL 1995*, pages 85–94. ACM, 1995.

[ES96] E. Allen Emerson and A. Prasad Sistla. Symmetry and model checking. *Formal Methods in System Design*, 9(1–2):105–131, 1996.

[Esp98] Javier Esparza. Decidability and complexity of Petri net problems: An introduction. In *Lectures on Petri Nets I: Basic Models*, volume 1491 of *Lecture Notes in Computer Science*, pages 374–428. Springer, 1998.

[Esp14] Javier Esparza. Keeping a crowd safe: On the complexity of parameterized verification. In *Proceedings of STACS 2014*, volume 25 of *Leibniz International Proceedings in Informatics*, pages 1–10. Schloss Dagstuhl, 2014.

[FKP15] Azadeh Farzan, Zachary Kincaid, and Andreas Podelski. Proof spaces for unbounded parallelism. In *Proceedings of POPL 2015*, pages 407–420. ACM, 2015.

[FQ03] Cormac Flanagan and Shaz Qadeer. Thread-modular model checking. In *Proceedings of SPIN 2003*, volume 2648 of *Lecture Notes in Computer Science*, pages 213–224. Springer, 2003.

[FS01] Alain Finkel and Philippe Schnoebelen. Well-structured transition systems everywhere! *Theoretical Computer Science*, 256(1–2):63–92, 2001.

[GR10a] Silvio Ghilardi and Silvio Ranise. Backward reachability of array-based systems by SMT solving: Termination and invariant synthesis. *Logical Methods in Computer Science*, 6(4), 2010.

[GR10b] Silvio Ghilardi and Silvio Ranise. MCMT: A model checker modulo theories. In *Proceedings of IJCAR 2010*, volume 6173 of *Lecture Notes in Computer Science*, pages 22–29. Springer, 2010.

[Gro96] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of STOC 1996*, pages 212–219. ACM, 1996.

[GS92] Steven M. German and A. Prasad Sistla. Reasoning about systems with many processes. *Journal of the ACM*, 39(3):675–735, 1992.

[GS97] Viktor Gyuris and A. Prasad Sistla. On-the-fly model checking under fairness that exploits symmetry. In *Proceedings of CAV 1997*, volume 1254 of *Lecture Notes in Computer Science*, pages 232–243. Springer, 1997.

[GSM16] Arie Gurfinkel, Sharon Shoham, and Yuri Meshman. SMT-based verification of parameterized systems. In *Proceedings of FSE 2016*, pages 338–348. ACM, 2016.

[Hig52] Graham Higman. Ordering by divisibility in abstract algebras. *Proceedings of the London Mathematical Society*, s3-2(1):326–336, 1952.

[HJJ+95] Jesper G. Henriksen, Jakob L. Jensen, Michael E. J. Jørgensen, Nils Klarlund, Robert Paige, Theis Rauhe, and Anders Sandholm. Mona: Monadic second-order logic in practice. In *Proceedings of TACAS 1995*, volume 1019 of *Lecture Notes in Computer Science*, pages 89–110. Springer, 1995.

[HMP17] Jochen Hoenicke, Rupak Majumdar, and Andreas Podelski. Thread modularity at many levels: A pearl in compositional verification. In *Proceedings of POPL 2017*, pages 473–485. ACM, 2017.

[HRH+21] Kesha Hietala, Robert Rand, Shih-Han Hung, Xiaodi Wu, and Michael Hicks. A verified optimizer for quantum circuits. *Proceedings of the ACM on Programming Languages*, 5(POPL):1–29, 2021.

[HV05] Peter Habermehl and Tomáš Vojnar. Regular model checking using inference of regular languages. *Electronic Notes in Theoretical Computer Science*, 138(3):21–36, 2005.

[ID96] C. Norris Ip and David L. Dill. Better verification through symmetry. *Formal Methods in System Design*, 9(1–2):41–75, 1996.

[JKS+13] Annu John, Igor Konnov, Ulrich Schmid, Helmut Veith, and Josef Widder. Parameterized model checking of fault-tolerant distributed algorithms by abstraction. In *Proceedings of FMCAD 2013*, pages 201–209. IEEE, 2013.

[KBI+17] Aleksandr Karbyshev, Nikolaj Bjørner, Shachar Itzhaky, Noam Rinetzky, and Sharon Shoham. Property-directed inference of universal invariants or proving their absence. *Journal of the ACM*, 64(1):1–33, 2017.

[KGMC22] Shankaranarayanan Krishna, Adwait Godbole, Roland Meyer, and Soham Chakraborty. Parameterized verification under release acquire is PSPACE-complete. In *Proceedings of PODC 2022*, pages 482–492. ACM, 2022.

[KKW10] Alexander Kaiser, Daniel Kroening, and Thomas Wahl. Dynamic cutoff detection in parameterized concurrent programs. In *Proceedings of CAV 2010*, volume 6174 of *Lecture Notes in Computer Science*, pages 645–659. Springer, 2010.

[KLVW17] Igor Konnov, Marijana Lazić, Helmut Veith, and Josef Widder. A short counterexample property for safety and liveness verification of fault-tolerant distributed algorithms. In *Proceedings of POPL 2017*, pages 719–734. ACM, 2017.

[KM89] Robert P. Kurshan and Kenneth L. McMillan. A structural induction theorem for processes. In *Proceedings of PODC 1989*, pages 239–247. ACM, 1989.

[KMM+97] Yonit Kesten, Oded Maler, Monica Marcus, Amir Pnueli, and Elad Shahar. Symbolic model checking with rich assertional languages. In *Proceedings of CAV 1997*, volume 1254 of *Lecture Notes in Computer Science*, pages 424–435. Springer, 1997.

[KMM+01] Yonit Kesten, Oded Maler, Monica Marcus, Amir Pnueli, and Elad Shahar. Symbolic model checking with rich assertional languages. *Theoretical Computer Science*, 256(1–2):93–112, 2001.

[KPIA20] Jason R. Koenig, Oded Padon, Neil Immerman, and Alex Aiken. First-order quantified separators. In *Proceedings of PLDI 2020*, pages 703–717. ACM, 2020.

[KPSZ02] Yonit Kesten, Amir Pnueli, Elad Shahar, and Lenore Zuck. Network invariants in action. In *Proceedings of CONCUR 2002*, volume 2421 of *Lecture Notes in Computer Science*, pages 101–115. Springer, 2002.

[KVW17] Igor Konnov, Helmut Veith, and Josef Widder. On the completeness of bounded model checking for threshold-based distributed algorithms: Reachability. *Information and Computation*, 252:95–109, 2017.

[Lam94] Leslie Lamport. The temporal logic of actions. *ACM Transactions on Programming Languages and Systems*, 16(3):872–923, 1994.

[Lam02] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[LB07] Shuvendu K. Lahiri and Randal E. Bryant. Predicate abstraction with indexed predicates. *ACM Transactions on Computational Logic*, 9(1), 2007.

[LHR97] David Lesens, Nicolas Halbwachs, and Pascal Raymond. Automatic verification of parameterized linear networks of processes. In *Proceedings of POPL 1997*, pages 346–357. ACM, 1997.

[LR16] Anthony W. Lin and Philipp Rümmer. Liveness of randomised parameterised systems under arbitrary schedulers. In *Proceedings of CAV 2016*, volume 9779 of *Lecture Notes in Computer Science*, pages 112–133. Springer, 2016.

[Mar94] Alberto Marcone. Foundations of BQO theory. *Transactions of the American Mathematical Society*, 345(2):641–660, 1994.

[McM01] Kenneth L. McMillan. Parameterized verification of the FLASH cache coherence protocol by compositional model checking. In *Proceedings of CHARME 2001*, volume 2144 of *Lecture Notes in Computer Science*, pages 179–195. Springer, 2001.

[MGJ+19] Haojun Ma, Aman Goel, Jean-Baptiste Jeannin, Manos Kapritsos, Baris Kasikci, and Karem A. Sakallah. I4: Incremental inference of inductive invariants for verification of distributed protocols. In *Proceedings of SOSP 2019*, pages 370–384. ACM, 2019.

[NJ13] Daniel Neider and Nils Jansen. Regular model checking using solver technologies and automata learning. In *Proceedings of NFM 2013*, volume 7871 of *Lecture Notes in Computer Science*, pages 16–31. Springer, 2013.

[NRZ+15] Chris Newcombe, Tim Rath, Fan Zhang, Bogdan Munteanu, Marc Brooker, and Michael Deardeuff. How Amazon web services uses formal methods. *Communications of the ACM*, 58(4):66–73, 2015.

[OG76] Susan Owicki and David Gries. An axiomatic proof technique for parallel programs I. *Acta Informatica*, 6(4):319–340, 1976.

[OSS09] Scott Owens, Susmit Sarkar, and Peter Sewell. A better x86 memory model: x86-TSO. In *Proceedings of TPHOLs 2009*, volume 5674 of *Lecture Notes in Computer Science*, pages 391–407. Springer, 2009.

[PLSS17] Oded Padon, Giuliano Losa, Mooly Sagiv, and Sharon Shoham. Paxos made EPR: Decidable reasoning about distributed protocols. *Proceedings of the ACM on Programming Languages*, 1(OOPSLA):1–31, 2017.

[PMP+16] Oded Padon, Kenneth L. McMillan, Aurojit Panda, Mooly Sagiv, and Sharon Shoham. Ivy: Safety verification by interactive generalization. In *Proceedings of PLDI 2016*, pages 614–630. ACM, 2016.

[PRZ01] Amir Pnueli, Sitvanit Ruah, and Lenore Zuck. Automatic deductive verification with invisible invariants. In *Proceedings of TACAS 2001*, volume 2031 of *Lecture Notes in Computer Science*, pages 82–97. Springer, 2001.

[PXZ02] Amir Pnueli, Jessie Xu, and Lenore Zuck. Liveness with $(0, 1, \infty)$-counter abstraction. In *Proceedings of CAV 2002*, volume 2404 of *Lecture Notes in Computer Science*, pages 107–122. Springer, 2002.

[Rac78] Charles Rackoff. The covering and boundedness problems for vector addition systems. *Theoretical Computer Science*, 6(2):223–231, 1978.

[RMV22] Azalea Raad, Luc Maranget, and Viktor Vafeiadis. Extending Intel-x86 consistency and persistency: Formalising the semantics of Intel-x86 memory types and non-temporal stores. *Proceedings of the ACM on Programming Languages*, 6(POPL):1–31, 2022.

[SG99] A. Prasad Sistla and Viktor Gyuris. Parameterized verification of linear networks using automata as invariants. *Formal Aspects of Computing*, 11(4):402–425, 1999.

[Sho97] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.

[SSO+10] Peter Sewell, Susmit Sarkar, Scott Owens, Francesco Zappa Nardelli, and Magnus O. Myreen. x86-TSO: A rigorous and usable programmer's model for x86 multiprocessors. *Communications of the ACM*, 53(7):89–97, 2010.

[Suz88] Ichiro Suzuki. Proving properties of a ring of finite-state machines. *Information Processing Letters*, 28(4):213–214, 1988.

[Tou01] Tayssir Touili. Regular model checking using widening techniques. *Electronic Notes in Theoretical Computer Science*, 50(4), 2001.

[TT08] Muralidhar Talupur and Mark R. Tuttle. Going with the flow: Parameterized verification using message flows. In *Proceedings of FMCAD 2008*, pages 1–8. IEEE, 2008.

[WB98] Pierre Wolper and Bernard Boigelot. Verifying systems with infinite but regular state spaces. In *Proceedings of CAV 1998*, volume 1427 of *Lecture Notes in Computer Science*, pages 88–97. Springer, 1998.

[WL89] Pierre Wolper and Vinciane Lovinfosse. Verifying properties of large sets of processes with network invariants. In *Proceedings of the International Workshop on Automatic Verification Methods for Finite State Systems*, volume 407 of *Lecture Notes in Computer Science*, pages 68–80. Springer, 1989.

[Yin11] Mingsheng Ying. Floyd–Hoare logic for quantum programs. *ACM Transactions on Programming Languages and Systems*, 33(6):1–49, 2011.

[ZP04] Lenore Zuck and Amir Pnueli. Model checking and abstraction to the aid of parameterized systems (a survey). *Computer Languages, Systems & Structures*, 30(3–4):139–169, 2004.